

Grad Café SQL Analysis: Query Results

JHU EN.605.256 Modern Software Concepts in Python · Module 3 · generated September 14, 2026 12:37

Database: 30,502 entries added between January 15, 2026 and September 14, 2026.

Valid-range rule for averages: a score counts as provided only on its official scale (GPA above 0 and at most 4; GRE Verbal and Quantitative 130–170; Analytical Writing 0–6). In this database 1,459 values in the GRE Quantitative column are off that scale, 1,427 of them 260–340 composite totals; averaging the column as stored would give 259.96. Also excluded: 84 Analytical Writing values and 135 GPAs off their scales. Stored values are never modified.

Matching conventions: text comparisons ignore capitalization and surrounding spaces; an acceptance is a status starting with "accept"; regular expressions use PostgreSQL's ~* (case-insensitive) operator, where \m and \M mark word boundaries.

Question 1

How many entries in the database are from applicants who applied for Fall 2026?

RESULT

Fall 2026 applicant count: **30,066**

SQL QUERY (AS EXECUTED BY QUERY_DATA.PY)

```
SELECT COUNT(*) AS fall_2026_entries
FROM applicants
WHERE LOWER(TRIM(term)) = 'fall 2026';
```

WHAT THE QUERY DOES

Counts every row whose term, ignoring capitalization and surrounding spaces, is 'Fall 2026'. The term column is the intended start term shown on each Grad Café entry.

Question 2

Among entries that provide a nationality classification, what percentage are international students?

RESULT

Percent international: **46.35%**
International entries: 13,885
Entries with a nationality classification: 29,954

SQL QUERY (AS EXECUTED BY QUERY_DATA.PY)

```
SELECT
    COUNT(*) FILTER (
        WHERE LOWER(TRIM(us_or_international)) = 'international'
    ) AS international_entries,
    COUNT(*) AS classified_entries,
    ROUND(100.0 * COUNT(*) FILTER (
        WHERE LOWER(TRIM(us_or_international)) = 'international'
    ) / NULLIF(COUNT(*), 0), 2) AS percent_international
FROM applicants
WHERE LOWER(TRIM(us_or_international)) IN ('international', 'american', 'other');
```

WHAT THE QUERY DOES

The WHERE clause keeps only rows with a usable classification (International, American or Other), so missing and blank values are excluded from the denominator. FILTER counts the International rows among them, and the ratio is multiplied by 100 and rounded to two decimals. American and Other are part of the denominator but not the numerator. NULLIF avoids dividing by zero.

Question 3

What are the average GPA, GRE Quantitative, GRE Verbal, and GRE Analytical Writing scores of applicants who provide each metric?

RESULT

Average GPA: **3.77** (n = 18,287)
Average GRE Quantitative: 165.82 (n = 967)
Average GRE Verbal: 160.74 (n = 2,032)
Average GRE Analytical Writing: 4.35 (n = 1,829)

SQL QUERY (AS EXECUTED BY QUERY_DATA.PY)

```
SELECT
  ROUND((AVG(gpa) FILTER (WHERE gpa > 0 AND gpa <= 4.0))::numeric, 2) AS avg_gpa,
  COUNT(gpa) FILTER (WHERE gpa > 0 AND gpa <= 4.0) AS gpa_n,
  ROUND((AVG(gre) FILTER (WHERE gre BETWEEN 130 AND 170))::numeric, 2) AS avg_gre_quant,
  COUNT(gre) FILTER (WHERE gre BETWEEN 130 AND 170) AS gre_quant_n,
  ROUND((AVG(gre_v) FILTER (WHERE gre_v BETWEEN 130 AND 170))::numeric, 2) AS avg_gre_verbal,
  COUNT(gre_v) FILTER (WHERE gre_v BETWEEN 130 AND 170) AS gre_verbal_n,
  ROUND((AVG(gre_aw) FILTER (WHERE gre_aw BETWEEN 0 AND 6))::numeric, 2) AS avg_gre_aw,
  COUNT(gre_aw) FILTER (WHERE gre_aw BETWEEN 0 AND 6) AS gre_aw_n
FROM applicants;
```

WHAT THE QUERY DOES

Each average has its own FILTER, so an applicant contributes to every metric they reported, independently of the other three. A value counts as reported only when it lies on the official scale: GPA above 0 and at most 4.0, GRE Verbal and Quantitative 130 to 170, Analytical Writing 0 to 6. Many applicants type their 260-340 GRE total into the Quantitative field, or 99.99 into Analytical Writing; including those would produce averages that are not scores at all. AVG ignores NULLs, and ROUND(...::numeric, 2) rounds to two decimals.

Question 4

What is the average GPA of American applicants who applied for Fall 2026?

RESULT

Average GPA of American Fall 2026 applicants: **3.79**
Applicants with a GPA: 10,354

SQL QUERY (AS EXECUTED BY QUERY_DATA.PY)

```
SELECT
    ROUND(AVG(gpa)::numeric, 2) AS avg_gpa,
    COUNT(gpa)                  AS applicants_with_gpa
FROM applicants
WHERE LOWER(TRIM(term)) = 'fall 2026'
    AND LOWER(TRIM(us_or_international)) = 'american'
    AND gpa > 0 AND gpa <= 4.0;
```

WHAT THE QUERY DOES

Restricts the rows to Fall 2026 entries classified as American that report a GPA on the 4.0 scale, then averages the GPA. COUNT(gpa) shows how many applicants the average is based on.

Question 5

What percentage of Fall 2025 entries are acceptances?

RESULT

Fall 2025 acceptance percentage: **48.22%**
Accepted Fall 2025 entries: 95
All Fall 2025 entries: 197

SQL QUERY (AS EXECUTED BY QUERY_DATA.PY)

```
SELECT
    COUNT(*) FILTER (WHERE TRIM(status) ILIKE 'accept%') AS accepted_entries,
    COUNT(*) AS fall_2025_entries,
    ROUND(100.0 * COUNT(*) FILTER (WHERE TRIM(status) ILIKE 'accept%')
          / NULLIF(COUNT(*), 0), 2) AS acceptance_percent
FROM applicants
WHERE LOWER(TRIM(term)) = 'fall 2025';
```

WHAT THE QUERY DOES

Takes all Fall 2025 entries as the denominator and counts, with FILTER, those whose cleaned status starts with 'accept' as the numerator. The percentage is rounded to two decimals.

Question 6

What is the average GPA of accepted applicants who applied for Fall 2026?

RESULT

Average GPA of accepted Fall 2026 applicants: **3.76**
Applicants with a GPA: 7,321

SQL QUERY (AS EXECUTED BY QUERY_DATA.PY)

```
SELECT
    ROUND(AVG(gpa)::numeric, 2) AS avg_gpa,
    COUNT(gpa)                  AS applicants_with_gpa
FROM applicants
WHERE LOWER(TRIM(term)) = 'fall 2026'
    AND TRIM(status) ILIKE 'accept%'
    AND gpa > 0 AND gpa <= 4.0;
```

WHAT THE QUERY DOES

Restricts the rows to Fall 2026 entries whose status indicates an acceptance and that report a GPA on the 4.0 scale, then averages the GPA.

Question 7

How many entries are from applicants who applied to Johns Hopkins University for a master's degree in Computer Science?

RESULT

JHU Computer Science master's entries: **8**

SQL QUERY (AS EXECUTED BY QUERY_DATA.PY)

```
SELECT COUNT(*) AS jhu_cs_masters_entries
FROM applicants
WHERE program ~* '(johns?\s+hopkins|mjhu\M)'
    AND program ~* '(computer\s+science|meecs\M)'
    AND degree ~* '^s*master';
```

WHAT THE QUERY DOES

Uses the original downloaded fields only. The program column holds the text 'Program, University', so one case-insensitive regular expression recognizes Johns Hopkins (also 'John Hopkins' and the acronym JHU as a whole word) and another recognizes Computer Science (also the EECS acronym). The degree column must start with 'Master'.

Question 8

How many Fall 2026 entries are acceptances from applicants applying for a PhD in Computer Science at Georgetown University, MIT, Stanford University, or Carnegie Mellon University (using the original downloaded fields)?

RESULT

Accepted Fall 2026 CS PhD entries (original fields): **32**

SQL QUERY (AS EXECUTED BY QUERY_DATA.PY)

```
SELECT COUNT(*) AS accepted_cs_phd_entries
FROM applicants
WHERE LOWER(TRIM(term)) = 'fall 2026'
  AND TRIM(status) ILIKE 'accept%'
  AND degree ~* '^s*ph\.\?s*d'
  AND program ~* '(computer\s+science|\meecs\M)'
  AND (   program ~* 'georgetown'
        OR program ~* 'massachusetts\s+institute\s+of\s+technology'
        OR program ~* '\mMIT\M'
        OR program ~* 'stanford'
        OR program ~* '(carnegie\s+mellon|\mcmu\M)');
```

WHAT THE QUERY DOES

Applies all five restrictions at once: term Fall 2026, an accepted status, a PhD degree (PhD, Ph.D.), a Computer Science program (including 'Electrical Engineering and Computer Science', which is how MIT lists its CS PhD, and the EECS acronym), and one of the four universities recognized in the original program text. MIT is matched both by its full name and by the case-sensitive acronym.

Question 9

Repeating Question 8 with the LLM-generated program and university fields, how does the count compare with the original-field count?

RESULT

Original-field count: **32**

LLM-field count: 32

Difference: 0

SQL QUERY (AS EXECUTED BY QUERY_DATA.PY)

```
SELECT
  COUNT(*) FILTER (
    WHERE program ~* '(computer\s+science|\meecs\M)'
    AND (
      program ~* 'georgetown'
      OR program ~* 'massachusetts\s+institute\s+of\s+technology'
      OR program ~* '\mMIT\M'
      OR program ~* 'stanford'
      OR program ~* '(carnegie\s+mellon|\mcmu\M)')
  ) AS original_field_count,
  COUNT(*) FILTER (
    WHERE llm_generated_program ~* '(computer\s+science|\meecs\M)'
    AND LOWER(TRIM(llm_generated_university)) IN ('georgetown university',
                                                    'massachusetts institute of technology',
                                                    'stanford university',
                                                    'carnegie mellon university')
  ) AS llm_field_count
FROM applicants
WHERE LOWER(TRIM(term)) = 'fall 2026'
AND TRIM(status) ILIKE 'accept%'
AND degree ~* '^s*ph\.\?\s*d';
```

WHAT THE QUERY DOES

Keeps the term, status and degree restrictions of Question 8 in the WHERE clause and computes both counts in one pass with FILTER: the first uses the original program text exactly as in Question 8, the second uses the LLM-generated program and the LLM-generated university, which must equal one of the four canonical university names. The difference is LLM count minus original count. Why the counts can agree: Grad Café's current submission form has applicants pick their university from a list, so these four schools already arrive with one spelling each (the LLM step only removes the '(MIT)' suffix), and their Computer Science program names are short and consistent, so both approaches select the same entries. The fields would diverge on free-text entries: a misspelled or unusually written school or program that the patterns above miss (for example 'Stanferd', 'Carnegie-Mellon', or 'CS' without the words 'Computer Science') is caught by the LLM field only when the standardizer maps it to the canonical name, while a hallucinated or over-merged LLM name can add or drop an entry that the original text classifies correctly.

Question 10

Original question: For Fall 2026, how do the acceptance rate and the average GPA of accepted applicants compare across degree types that have at least 100 entries?

RESULT

Degree	Entries	Acceptances	Acceptance %	Avg GPA (accepted)
PhD	21,396	5,569	26.03%	3.81
Masters	7,515	5,128	68.24%	3.72
MFA	602	191	31.73%	3.73
PsyD	309	108	34.95%	3.84
Other	208	122	58.65%	3.71

SQL QUERY (AS EXECUTED BY QUERY_DATA.PY)

```
SELECT
  degree,
  COUNT(*)                                AS entries,
  COUNT(*) FILTER (WHERE TRIM(status) ILIKE 'accept%')  AS acceptances,
  ROUND(100.0 * COUNT(*) FILTER (WHERE TRIM(status) ILIKE 'accept%')
        / NULLIF(COUNT(*), 0), 2)          AS acceptance_percent,
  ROUND((AVG(gpa) FILTER (WHERE TRIM(status) ILIKE 'accept%'
                          AND gpa > 0 AND gpa <= 4.0))::numeric, 2)  AS avg_accepted_gpa
FROM applicants
WHERE LOWER(TRIM(term)) = 'fall 2026'
  AND degree IS NOT NULL
GROUP BY degree
HAVING COUNT(*) >= 100
ORDER BY entries DESC, degree;
```

WHAT THE QUERY DOES

Groups Fall 2026 entries by degree type, keeps only groups with at least 100 entries (HAVING), and for each group computes the number of entries, the number and percentage of acceptances, and the average GPA of the accepted applicants who report a GPA on the 4.0 scale.

Question 11

Original question: Which ten universities (LLM-standardized names) have the most Fall 2026 entries, and what percentage of each university's entries report an acceptance?

RESULT

University	Entries	Acceptances	Acceptance %
University of Michigan, Ann Arbor	803	232	28.89%
Stanford University	740	140	18.92%
University of California, Berkeley	690	176	25.51%
Yale University	592	85	14.36%
Columbia University	578	285	49.31%
Princeton University	571	121	21.19%
Massachusetts Institute of Technology	561	102	18.18%
University of Washington	553	156	28.21%
University of Wisconsin–Madison	543	103	18.97%
Harvard University	535	109	20.37%

SQL QUERY (AS EXECUTED BY QUERY_DATA.PY)

```
SELECT
    llm_generated_university                AS university,
    COUNT(*)                               AS entries,
    COUNT(*) FILTER (WHERE TRIM(status) ILIKE 'accept%') AS acceptances,
    ROUND(100.0 * COUNT(*) FILTER (WHERE TRIM(status) ILIKE 'accept%')
          / NULLIF(COUNT(*), 0), 2)         AS acceptance_percent
FROM applicants
WHERE LOWER(TRIM(term)) = 'fall 2026'
    AND llm_generated_university IS NOT NULL
    AND llm_generated_university <> 'Unknown'
GROUP BY llm_generated_university
ORDER BY entries DESC, university
LIMIT 10;
```

WHAT THE QUERY DOES

Groups Fall 2026 entries by the LLM-standardized university name, which merges most spelling variants of the same school, ignores rows the standardizer could not attribute ('Unknown'), sorts by the number of entries and keeps the top ten, with the share of each school's entries that report an acceptance. A few small leftovers stay separate groups (for example 'Yale' or 'Stanford' written alone, each with fewer than ten rows); they are too small to change which schools make the top ten.